Enabling Efficient Communications with Session Multipathing

Israel Luiz Borges Ribeiro[1], **and Bruno Yuji Lino Kimura**[1]

[1]Instituto de Matemática e Computação – IMC
Universidade Federal de Itajubá – UNIFEI
Av. BPS, 1303, Pinheirinho, Itajubá-MG, CEP 37500-903, Brasil

{israelborges, kimura}@unifei.edu.br

Abstract. *In this paper we report an investigation on potential session multipathing strategies that leverage the easy deployment of user space-based protocols. We devised an experimental session layer architecture to deal with multipathing and we implemented such architecture with 4 different libraries of asynchronous I/O processing for Linux systems: Epoll, Posix Threads, Posix AIO, and Libev. We discuss the evaluation of these implementations (a) by comparing them with the general kernel space solution, Multipath TCP (MPTCP), in an emulated network; and (b) by determining both the performance gain factor and the cost of resource consumption as function of the number of paths in a session. We found that Libev API, a full-featured and high-performance event loop, applied to the session multipathing enables an average goodput gain factor of 1.62 faster per path added in a session, while its counterpart is of 2.23% of CPU utilization per path and it requires no more than 4 MB of RAM regardless the number of paths. We also observed that Libev-based multipathing allows overall efficiency slightly higher than MPTCP.*

1. Introduction

Multihoming, *network striping*, and *multiple paths* are resource aggregation techniques that take the advantage of multiple communication possibilities of a node, divide the application data into partitions, and stripe them over the several communication mediums, e.g. over multiple network interfaces and/or end-to-end sessions.

Several solutions have been proposed to deal with this concern. Carns *et. al* [Carns et al. 2005] proposed a Network Abstraction Layer for Parallel I/O. Yildirim *et. al* [Yildirim et al. 2009] analysed the results of different techniques to balance TCP buffer and parallel streams and present the initial steps to a balanced modelling of throughput based on optimized parameters. Parallel Sockets (PSockets) proposed by Sivakumar *et al* [Sivakumar et al. 2000] uses network striping to allow applications to achieve near optimal utilization of the network bandwidth without the necessity of tuning the TCP window size. Parallel Sockets proposed by Altman *et. al* [Altman et al. 2006] is generic hack to improve throughput attained by TCP for bulk data transfers by opening several TCP connections and striping the data file over them.

Although the use of multiple TCP paths in a session is not a novel idea [Hsieh and Sivakumar 2002] [Magalhaes and Kravets 2001], there is a lack of standardization that contemplates multipathing for today's network resources. Today's most of mobile devices support at least IEEE 802.11 and 3G/4G communication technologies

[Barré et al. 2010], while data centres can offer higher aggregate bandwidth and robustness by creating multiple paths in the core of the network [Raiciu et al. 2011a]. This increases the interest in using several access mediums in the same connection, so that it is possible to transparently change from one medium to another in case of failure, and simultaneously improve end-to-end throughput [Barré et al. 2010].

In this sense, MPTCP (Multipath TCP) [Ford et al. 2011] has become a standard protocol of eminent importance. It assumes that an end-system of an Internet node is able to establish multiple end-to-end reliable data streams (called *paths*) over a single or multiple network interfaces. The simultaneous use of multiple paths for a TCP/IP session improves resource usage within the network and, thus, (a) improve user experience through higher throughput, and (b) improve resilience to network failure [Ford et al. 2013]. The failure resilience intrinsically provides support for mobility, when the node migrates across networks on the Internet. It maintains more than one active link for as long as is feasible, if connectivity is lost for one path, remaining one can continue without interruption [Raiciu et al. 2011c].

MPTCP is essentially implemented in the OS' kernel. On one hand, kernel space-based protocols prevent context switching and memory copy between user and kernel spaces; on the other hand, they are difficult to deploy and maintain when requiring to unify the protocol implementation to the desired kernel code. This constraints are avoided with solutions implemented in the user space, which are software components made independently of kernel restrictions.

From our previous experience on development of session-based mobile communication solutions [Kimura et al. 2013] [Kimura et al. 2012], we take the hypothesis that strategies of asynchronous I/O processing can be used in support for multipath session management in the user space, so that such strategies could allow performance similar to the MPTCP. Session layer striping allows applications to take advantage of multihoming, while avoiding most of the deployability issues traditionally linked with modifying Application Layer code [Habib et al. 2006].

We propose a general multipath session layer architecture and we implemented it by using different techniques of I/O asynchronous processing for Linux system, such as Epoll[1], Posix Threads[2], Posix AIO[3], and Libev[4]. With an emulated network environment, we then compared these multipath techniques with MPTCP. To the best of our knowledge, unlike previous works, this paper provides (a) the cost-benefit analysis of multipathing by evaluating both the performance gain (goodput) and the resource consumption (CPU and memory) as function of the number of paths in a session; (b) the goodput gain factor for adding up a path in a session. In this paper we concerned with determining the boundary of multipathing gain and its resource utilization overhead, as well as with determining the most suitable technique for session multipathing, i.e. the one that provides the best cost-benefit. Provisioning of failure tolerance and mobility support are beyond the scope of this paper.

[1] `http://man7.org/linux/man-pages/man7/epoll.7.html`
[2] `http://man7.org/linux/man-pages/man7/pthreads.7.html`
[3] `http://man7.org/linux/man-pages/man7/aio.7.html`
[4] `http://linux.die.net/man/3/ev`

We observed that user space-based multipathing enables similar or better performance than MPTCP. Libev-based session multipathing has allowed high performance, with 1.62 goodput gain factor, in the meantime low resource consumption, with 2.23% of average CPU utilization per path and no more than 4MB of RAM. While MPTCP, in the same evaluated scenario, has provided 1.53 goodput gain factor, 4.85% of average CPU utilization, and 4-6MB of RAM.

This papers is organized as follows: next section presents details of the proposed multipath session architecture, with its main components: Connection Management, Message Scheduling, and Interface of Interpaths; Section 3 presents details of the implementation with thread-based and event-driven approaches; Section 4 discuss the results of performance gain and resource consumption of our multipath session implementations and MPTCP; and Section 5 brings our conclusions and future directions.

2. User space multipath sessions

Our general architecture to deal with session multipathing is illustrated in Figure 1. As in MPTCP, the components were designed to provide basic functionalities, such as:

- **Connection Management**: it recognizes the active network interfaces of the destination node and establishes TCP connections according to each interface IP address. We assume that an active network interface is the one that is associated with the local link, addressed by an IP, and has access routes to the Internet.
- **Message Scheduling**: it splits a message in chunks that are transmitted concurrently over the multiple paths.
- **Interface of Interpaths**: it manages the sending and receiving of message chunks by multiplexing and demultiplexing such chunks in orderly fashion, respectively.

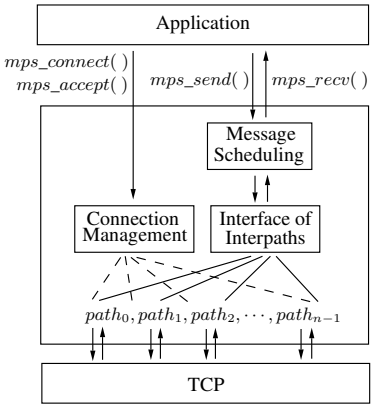

Figure 1. A general user space architecture for session multipathing.

The multipath session works as a middleware, which is an interface to the Application Layer implemented by means of a general purpose API. Such middleware provides for particular primitives that encapsulate the handling of multipathing, e.g. $mps\_connect(\ )$, $mps\_accept(\ )$, $mps\_send(\ )$, $mps\_recv(\ )$, etc. These primitives are based on the Socket API semantic in order to preserve the logic of the application code. We bring more details in next subsections.

2.1. Connection Management

We assume that the client is aware of at least one server's IP-port pair. Then it establishes a first connection with the server. Once established, the server returns to the client the IP addresses of all its active interfaces. For each server's active interface, the client then establishes a respective TCP connection. Thus, each path is consisted of a single TCP connection, so that the TCP Congestion Control works separately. Once established the TCP connections for all the paths, session establishment ends up.

An essential security mechanism is authenticating the peers involved in a session. Mutual challenge-response authentication between peers could prevent session hijacking, man-in-the-middle, and replay attacks [Kimura et al. 2013]. In addition, encryption mechanisms using symmetric and/or asymmetric keys, from the SSL API, could provide confidentiality between the involved nodes. Our implementations do not deal with security issues for while, and we leave them to be addressed in the future work.

2.2. Message Scheduling

Splitting the message in chunks according to the number of available paths is a task of the Message Scheduling component. The chunks are passed to the Interface of Interpaths component, which controls the sequence of the chunks transmitted over each single path. The Message Scheduling applies scatter-gather operations to handle the message chunks. Two abstract functions are used for that, respectively: $scatter(s, m, n)$, and $gather(s)$.

The abstract function $scatter(s, m, n)$ splits a message m of n bytes in length into a number of chunks as the number of paths available in the session s. The scatter function works according to Algorithm 1, which is used for both sending and receiving procedures.

Algorithm 1: $scatter(s, m, n)$: disjoining the message in chunks.

Input: A session s; a buffer m that stores the message; the total bytes n of the message to be sent or received.
Output: The setup of the session s.

```
1  begin
2      chunksize ← n / s.npaths;
3      for i ← 0 to s.npaths − 1 do
4          s.path[i].buf ← m + i × chunksize;
5          s.path[i].cbytes ← 0;
6          if i < s.npaths − 1 then
7              s.path[i].nbytes ← chunksize;
8          else
9              s.path[i].nbytes ← chunksize + (n −
               s.npaths × chunksize);
10         end
11     end
12     return s;
13 end
```

Algorithm 2: $gather(s)$: joining the message from chunks.

Input: A session s under asynchronous transmission.
Output: the total successfully nc bytes sent or received asynchronously.

```
1  begin
2      nc ← 0;
3      wait_async_return(s);
4      for i ← 0 to s.npaths − 1 do
5          nc ← nc + s.path[i].cbytes;
6          s.path[i].nbytes ←
           s.path[i].nbytes − s.path[i].cbytes;
7          s.path[i].buf ←
           s.path[i].buf + s.path[i].cbytes;
8          s.path[i].cbytes ← 0;
9      end
10     return nc;
11 end
```

For the sake of transmission consistency, the scatter algorithm handles the situation when the message splitting results in non-integer chunks. We consider that $chunksize$ is an integer variable will get always the integer part of the division of the message length (n) by the number of paths available ($s.npaths$). In the case of non-integer chunks, remaining fractional parts are computed for the last available path, which handles a chunk with one byte more.

The abstract function of $gather(s)$, described in Algorithm 2, attempts to synchronize the asynchronous transmissions. To do so, another abstract function, called

$wait\_async\_return()$, works as a logical barrier by waiting for the return of the initiated asynchronous operations that eventually are running.

After returning, the gather function scans through all the paths and updates the transmission context of each one. For each path, it adds up the number of the current bytes nc successfully transmitted; updates the value that will be transmitted $nbytes$ according to the amount already transmitted $cbyte$; then it updates the reference pointer in the buffer buf according to the amount of already transmitted bytes.

2.3. Interface of Interpaths

Two abstract functions are used for sending and receiving messages over the available paths in a session: $mps\_send(s, m, n)$ e $mps\_recev(s, m, n)$.

The function $mps\_send(s, m, n)$ is described in Algorithm 3. It applies the scatter algorithm to assign the portion of the message to be handled in each path. The function attempts to send chunks of the message while the amount of bytes ns is less than the amount of bytes n that should be sent. The asynchronous sending is conducted by the function $async\_send()$, which is a non-blocking function that uses the socket descriptor fd of the path i to accomplish the asynchronous sending. On error, a negative value is returned from the send function.

Algorithm 3: $mps\_send(s, m, n)$: multiplexer of message chunks over the multipaths.

Input: A session s; a buffer m that stores the message to be sent; the total n bytes of the message.
Output: Total bytes $ns = n$ asynchronously and successfully sent, or a negative value, on error.

```
1  begin
2      ns ← 0;
3      scatter(s, m, n);
4      while ns < n do
5          for i ← 0 to s.npaths − 1 do
6              r ← async_send(s.path[i].fd, m);
7              if r is failure then
8                  return −1;
9              end
10         end
11         ns ← ns + gather(s);
12     end
13     return ns;
14 end
```

Algorithm 4: $mps\_recv(s, m, n)$: demultiplexer of message chunks from the multipaths.

Input: A session s; a *buffer* m that will store the incoming message; n bytes of the of the message to b received.
Output: On success, the total bytes $nr \leq n$ asynchronously received, or a negative um valuer, on error.

```
1  begin
2      nr ← 0;
3      scatter(s, m, n);
4      while nr < n do
5          for i ← 0 to s.npaths − 1 do
6              r ← async_recv(s.path[i].fd, m);
7              if r is failure then
8                  return −1;
9              end
10         end
11         nr ← nr + gather(s);
12     end
13     return nr;
14 end
```

The non-blocking operation of $async\_send()$ allows for scanning all the paths without holding the subsequent processing. After the first scanning, then the current value of ns bytes is updated by adding up the bytes successfully sent. This amount is returned from the function $gather()$, described in Algorithm 2, which works as logical barrier to synchronize the sending. If one or more chunks remain to be sent, this procedure is repeated until the ns bytes reach the expected n ones, or upon an error during the sending.

The asynchronous receiving works according to Algorithm 4. Message receiving is also based on the steps of scatter-gather. The abstract function $async\_receive()$ provides for the use of the socket descriptor fd ready for receiving asynchronously. Upon receiving the bytes on the paths, the function $gather()$ updates the total nr received bytes and synchronizes the receiving. This procedure is repeated until the receipt of the message

is finished, or discontinued upon an error.

3. Implementation with asynchronous I/O techniques

The algorithms presented in the previous section are abstract functions of general purpose. They were implemented in Linux system by using asynchronous I/O processing according two strategies: *thread-based* and *event-driven*.

3.1. Thread-based session multipathing

We implemented the thread-based multipathing according to the dispatcher-worker model, using Posix Threads API. The dispatcher threads are represented by the abstract functions of $mps\_send()$ and $mps\_recv()$. The multiple worker threads, each one handling its path i, are instantiated during the session establishment. Figure 2 illustrates the life cycle of dispatchers and workers for the proposed architecture.

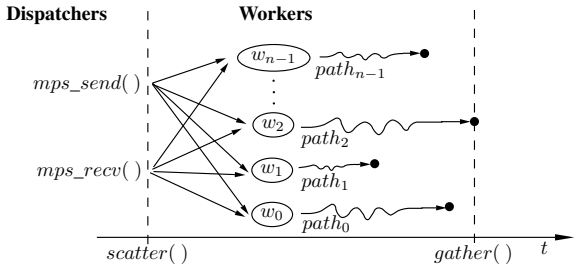

Figure 2. Life cycle of dispatcher-worker threads for session multipathing.

The proposed architecture provides for different buffers of sending and receiving, and there are no direct sharing of these resources. A worker thread i, works exclusively on a path i, so that there is no need for mutual exclusion during the concurrent operations of the threads. When adding a path in the session, a respective worker thread is created to handle the new path.

In $scatter$, dispatchers $mps\_send()$ and $mps\_recv()$ define which workers will handle the transmission of each chunk of the message m. All worker threads send and receive message chunks until the synchronization point in the $gather$. We implemented the barrier with functions from the **Posix Threads** API. The dispatchers initialize a barrier object with the function `pthread_barrier_init()`, which defines the number of worker threads will be blocked when reaching a synchronization point. The synchronization point is particularly represented by the abstract function $wait\_async\_return()$, line 3 of Algorithm 2. This function is provided by `pthread_barrier_wait()`, from POSIX Threads API.

Posix AIO is another thread-based API that provides asynchronous I/O. The I/O operations are governed by control blocks, called `aiocb`, which are a data structure used to store relevant information about the asynchronous transmission, e.g. the socket descriptor, a buffer and its offset, the size of the data to be transmitted, and the opcode for the asynchronous operation to be performed (`LIO_WRITE` or `LIO_READ`). After the step of $scatter$, the function `lio_listio()` is called to start executing the asynchronous transmission. With `LIO_WAIT` parameter, this function allows to configure the blocking operation, in which the calling thread is blocked until the end of all the started asynchronous

operations. Then, it provides a synchronization barrier. To obtain the return of asynchronous operations during the $gather$, the API provides the function `aio_return()`.

3.2. Event-driven session multipathing

Handling asynchronously I/O events is a technique to boost fast server's responses to many client requests. We used this technique in support for session multipathing in both client and server, as Figure 3 illustrates.

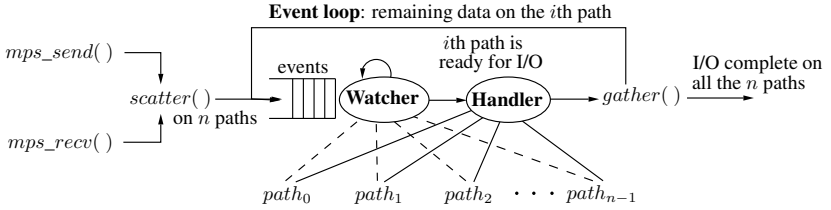

Figure 3. Life cycle of event-driven session multipathing.

The event-driven technique allows for a **watcher** that observers a queue of desired events over a poll of socket descriptors. In the proposed architecture, a socket descriptor belongs to a particular session. As soon as events occur on one or more descriptors, e.g. ready for reading or ready for writing, the watcher notifies an **event handler** about the state of an observed descriptor. Then, the event handler attempts to accomplish the input or output operations on the ready descriptor. The event handler is a single thread that makes use of non-blocking I/O operations, hence, allowing asynchronous handling and concurrency. In the proposed architecture, event handlers are represented by the abstract functions $async\_send()$ e $async\_recv()$.

If an I/O operation has not fully accomplished, depending on the asynchronous I/O technique, the desired event can persist or even be reinserted in the event queue. This allows an **event loop**, while there exist remaining data to be transmitted. According to Figure 3, we implemented the event-driven strategies for the proposed architecture using Epoll and Libev.

Epoll is a scalable API for monitoring multiple non-blocking socket descriptors to check if I/O is possible on any of them. It provides computational complexity of $O(1)$ as the number of file descriptors increase. Due to its better scalability, Epoll replaces the older ones, Posix Select and Posix Poll. In the proposed architecture, events are observed to deal with operations of sending (`EPOLLOUT`) and receiving (`EPOLLIN`). I/O operations on particular file descriptors are registered with the function `epoll_ctl()`, which is performed once during the $scatter$. The API provides the function `epoll_wait()` to wait for I/O events, which blocks the calling thread if no events are currently available. When events are available, the poll of socket descriptors are scanned to find the ones ready to accomplish the desired I/O operations. The event loop in Figure 3 is performed by combining: the event notification from `epoll_wait()`, the asynchronous I/O operation, and the $gather$.

Libev is an API for high performance full-featured event loop. The API provides the function `ev_io_init()` to initialize an watcher to observe a descriptor and to set the respective event handler. The event handler is implemented through an arbitrary call-back function, e.g. $async\_send()$ or $async\_recv()$, which is activated when occurring the

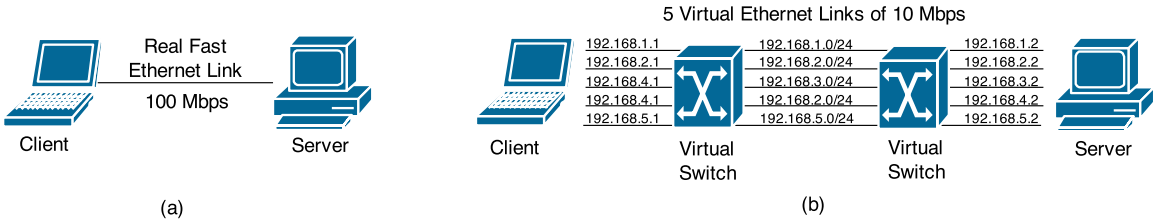

Figure 4. Network topology of tests: (a) real network topology, and (b) virtualized network topology.

VM	Processor	RAM	Disk	Host OS	Virtual OS
Server	Core i5	6 GB	1TB	Windows 7	Ubuntu 12.04 LTS
Client	Dual Core	1GB	320GB	Debian Squeeze	Debian Squeeze

Table 1. Hardware-software configuration of the emulated scenario with VMware.

desired event. The watcher can be stopped by the function `ev_io_stop()`. The synchronization occurs similarly to a barrier for threads, which is implemented by the function `ev_loop()`. This function would be equivalent to the abstract one $wait\_async\_return()$, in $gather$. During the event loop, all the initialized watchers check the pending events for their descriptors and activate the callbacks to handle them.

4. Obtained Results

We conducted experiments in emulated network to evaluate multipathing solutions as function of the number of paths in a session. We considered two aspects:

i. **Performance Gain** (goodput) with resource aggregation.
ii. **Cost of resource consumption** of memory and CPU.

4.1. Emulated Scenario

The choice of an emulated scenario rather than a real one was due to the easy implementation of a fully controlled test environment, free from noise and interference, besides the low cost of deployment.

The experiments were executed with the emulator VMware[5]. The emulated network topology is illustrated in Figure 4(b), which was virtualized on the real topology illustrated in Figure 4(a). The nodes Client and Server are virtual machines (VM) emulated in two different hosts. Table 1 presents the hardware-software configuration details.

We connected the machines point-to-point with a Fast Ethernet link, which is limited in 100 Mbps. Then, over this real topology, we created a virtual one. VMware allows to virtualize multiple collision domains with bridge configurations. Then, we created virtual switches so that the real Fast Ethernet link was divided into 5 virtual Ethernet links, each one limited in 10 Mbps. In both client and server, for each virtual link, the respective virtual network interface controller was connected in the virtual switch. There was an IP network for each virtual collision domain, as shows Figure 4(b).

Although the network topology utilized to evaluate multipathing is simple, it consists of disjoint and isolated links which allow each path to flow on a particular link

[5]`http://www.vmware.com/`

without compete to each other. This is a suitable scenario to obtain the top performance that multpathing protocols can get, since they work better over alternative paths like these. Once our focus in this paper is to determine the boundary of performance gain, we did not concern about using links of different capacities, parametrizing errors and latencies, or path selection.

4.2. Methodology

To evaluate MPTCP we load a VM with its Linux Kernel implementation (Kernel 3.5 MPTCP v0.88[6]). We implemented 5 versions of the proposed architecture, each one is named according to the used technology: Epoll, Epoll64, Threads, Posix AIO, and Libev. Epoll64 is implemented with Epoll technique, however, it is based on a variant $scatter$. When dividing the message by the number of available paths, we give the chunk of the message to be transmitted on the path. In Epoll64, the message chunk is fragmented in small sub-chunks of 64 KB in length regardless the number of paths. We intended to investigate the influence of transmitting small chunks in a multipath session.

To evaluate the implementations, an ordinary client-server application was created to use the access diversity of 5 virtual links. In this application, the client sends messages of varying length to the server by using a multipath session. The message size varies from 2KB and increases exponentially up to 1MB. The multipath session was established between the nodes, where each path is a TCP connection over the respective physical and logical network. In this scenario, experiments were executed to evaluate the performance gain and the cost of resource consumption. For each message size transmitted between the client and server, we collected at least 140 measurements of transmission latency, CPU utilization, and memory consumption.

4.3. The gain of resource aggregation

We aim to obtain the goodput of different techniques for multipathing by varying the number of paths in a session. To do so, we collected on the client side the latency for sending messages to the server and the respective message sizes.

From the results in Figure 5, we did not observe significant variation on the goodput of different message sizes, except for Epoll-based implementations that did provide apparent variable behaviour when changing the message size. Epoll is based on non-blocking I/O, so that every single I/O operation requires the application to poll descriptors to check the ones ready for reading and/or writing. Although we provide for a barrier synchronization, the readiness of the polled descriptors varies according to the demand for I/O operations and the respective OS management. We also observed that small message chunks do not resonate on the goodput, since Epoll and Epoll64 provide similar performance, as shows Figure 5.

The average goodput grows linearly as the number of paths, as show the dashed dot-lines over the boxplots in Figure 6(a). Figure 6(b) presents the gain factor when comparing a session of n paths with respect to the gain of a session of $n-1$ paths. The gain factor g_f is calculated as the ratio:

$$g_f = \frac{\bar{g_n}}{\bar{g_{n-1}}}, \tag{1}$$

[6]Available in: `http://multipath-tcp.org/`

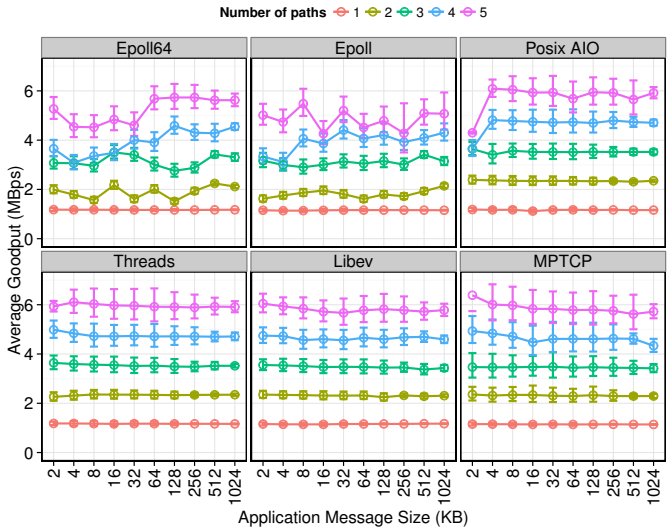

Figure 5. Average Goodput for I/O technique and number of paths, both as function of the message size.

where $\bar{g_n}$ and $\bar{g_{n-1}}$ are the average goodputs of sessions that hold n and $n-1$ paths, respectively.

Except for Epoll-based implementations, which have variable behaviour due to the polling operation accomplished by the application, we observed that the gain factor is higher when the number of paths increases from one to two, and the factor decreases from two to three paths, and so on. The gain factor decreases when the number of paths increases because the overhead to manage multiple paths also increases as the number of paths, so that such overhead impacts on the performance.

From the boxplots in Figure 6(a), we observed that MPTCP provides similar performance than the user space-based solutions, such as Posix AIO, Posix Threads, and Libev. Although MPTCP works at the kernel space and prevents context switching during I/O to the Application Layer, the transmission capacity of all paths is combined and managed together by the Coupled Congestion Control [Raiciu et al. 2011b]. On the other hand, user space-based multipath solutions have multiple TCP connections, so that each path capacity is managed separately by the ordinary TCP Congestion Control, and the goodput is then the sum of the capacity of each path. However, MPTCP congestion control improves throughput in a such way that the multipath flow should perform at least as well as a single path flow would on the best of the paths available [Raiciu et al. 2011b]. In addition, each path flows separately in a virtual ethernet link in our emulated network topology, so that there is no competition among TCP flows.

The most efficient technique was Libev, enabling a gain factor of 1.62 on average per additional path in a session. Epoll enables the lowest gain, increasing the goodput by a gain factor of 1.45 on average. Second lowest one was Epoll64, with 1.46 of gain factor on average. Worker-dispatcher thread-based multipathing and Posix AIO, both ones use the support of Posix Threads, provide very similar performance, with 1.53 and 1.52 of gain factor on average, respectively. Finally, MPTCP provides a gain of a factor of 1.53 per path.

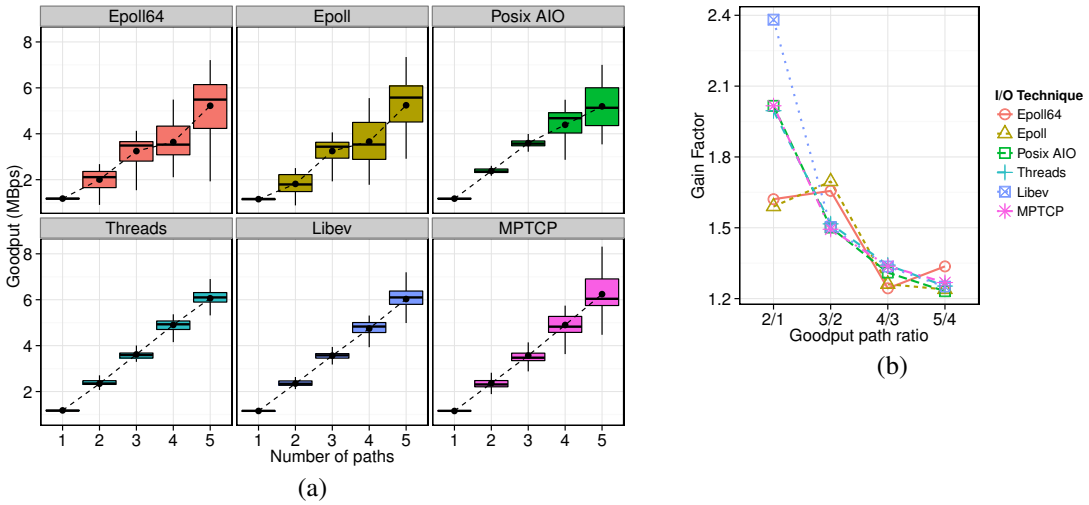

Figure 6. Analysis of Goodput: (a) boxplot for each I/O technique as function of the number of paths; (b) goodput gain factor g_f as function of the path ratio.

4.4. Resource consumption

Given the gain of resource aggregation, we assessed the cost of the server's session multipathing in terms of memory consumption and CPU utilization. We collected the measurements at the server side, since the server is the entity that provides the service, hence, the one that has its resource consumed by the demand of client requests.

Although CPU and RAM are resources cheap today, mobile devices (from smartphones to mobile robots) are still of limited resource capacity, mostly energy, and the occupations of CPU and RAM impact directly on the node's energy consumption.

4.4.1. Memory Consumption

The current RAM consumption information of the server process was obtained from the file `/proc/[pid]/status`[7] - given by the variable `VmSize`. Figure 7 illustrates the amplitude of server memory consumption as function of the number of paths. The largest memory consumers were the multipath techniques based on Posix Threads, such as our Threads and Posix AIO. The results show the greater the number of paths in a session, the greater the number of workers managed by a dispatcher, hence, the greater the expansion of the *memory heap* to handle multiples stack for different thread control blocks. Maintaining multiple threads to perform I/O is expensive and scales poorly[8] and other alternative models of I/O, such as event-driven, are often preferable [Kerrisk 2010].

Event-driven multipath techniques are quite lightweight with respect to memory consumption, since their event handlers are based on single-thread. This is confirmed by the results in Figure 7. The amplitudes of memory consumption of event-driven multipath ranged from 2MB to 5MB for Epoll and Libev. Epoll64, with its 64KB fragments of message, has required 10 to 12 MB of memory.

[7] `http://linux.die.net/man/5/proc`
[8] `http://man7.org/linux/man-pages/man7/aio.7.html`

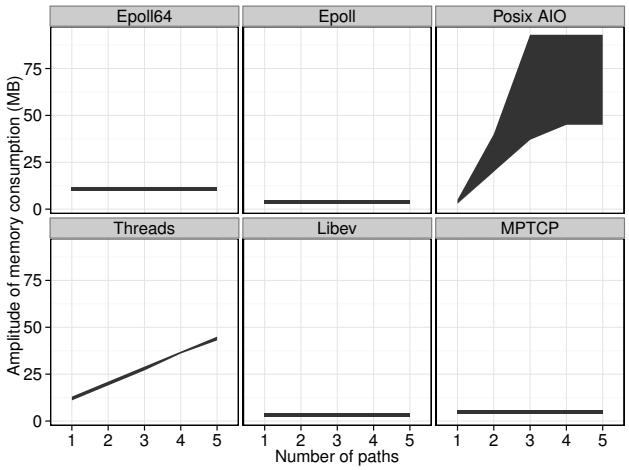

Figure 7. Amplitude of memory consumption as function of the number of paths.

4.4.2. CPU Utilization

The CPU usage reported by the VM's OS is relative to the CPU resources allocated to it by the host that virtualizes it. Therefore, for individual processes running on the VM's OS, the CPU utilization of such processes should still accurately reflect the relative proportion of the CPU time on the VM's OS.

We used `pidstat`[9] to obtain CPU utilization. This tool allows for monitoring individual tasks managed by the Linux Kernel. Similarly to the memory consumption test, we collected the server's CPU utilization while it remained running during the experiments.

We observed that Epoll and Epoll64 have required significant use of CPU, obtaining CPU utilization greater than 50% on average. Both techniques have consumed more kernel level CPU than any other. This is because of Epoll's event management uses mechanisms and facilities of the kernel for event notification. Epoll operates as a configurable kernel object in the user space, so that migrating from user to kernel mode is costly. Epoll64 often deliveries to the Transport Layer buffers (placed in the kernel space) application data containing the sub-blocks of 64KB. This implies more system calls, context switching, kernel event notifications, and, therefore, more CPU utilization.

The CPU utilization increases as the number of paths, as shows Figure 8. Except for Epoll-based techniques, there was little variation in consumption among techniques, so that Posix AIO, Posix Threads, Libev and MPCTP require similar CPU utilization. The average cost of adding a path in a session is 3.61%, 4.40%, 2.23% and 4.85% of CPU utilization for Posix AIO, Threads, Libev, MPTCP, respectively.

5. Conclusions and future directions

Finding out ways by which resources available in computer networks are fully utilized, with minimal or no impact on the existing logical and physical network infrastructure, is a big challenge when users and services on the Internet are constantly growing. Changes

[9] `http://linux.die.net/man/1/pidstat`

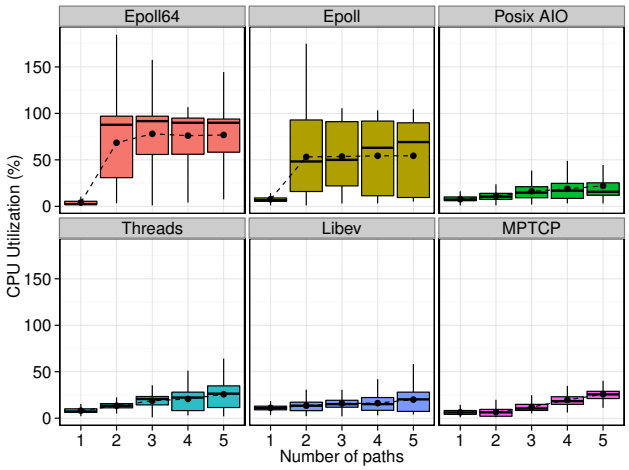

Figure 8. CPU utilization as function of the number of paths.

in already deployed infrastructures result in different costs, mostly in financial and operational spectrum.

When looking at the problematic of protocol implementation under the perspective of the general assumptions about OS' memory address spaces, a dilemma arises: we have to renounce ease of deployment and maintenance to obtain better performance, or vice-versa. In this paper, we propose multipath sessions that attempt to leverage the benefits of these two worlds. We provide (a) an investigation on different techniques to implement user space session multipathing in Linux systems, (b) an analysis on per path gain, (c) its counterpart measured through resource consumption, and (d) we point out the most suitable user space asynchronous IO technique that provides the best cost/benefit. We validate our research hypothesis when comparing experimental results of session multipathing with the ones of the general kernel space solution, MPTCP. We found that user space multipathing was able to provide performance slightly better than the kernel space one.

We consider the suitable session multipathing technique the one that provides lower resource consumption and, in the meantime, performance gain with higher goodput. From the results obtained in experiments in emulated environments, we point out Libev as the best cost-benefit, with higher performance and lower resource consumption. We also observed that Epoll and Threads were prohibitive techniques for session multipathing. Epoll-based multipathing requires a lot of CPU utilization, while Thread-based multipathing requires excessive memory.

Security issues involved in the session multipathing, particularly the session establishment, require further study. We also address in future works the analysis of energy consumption as function of the number of paths in session.

References

Altman, E., Barman, D., Tuffin, B., and Vojnovic, M. (2006). Parallel TCP Sockets: Simple Model, Throughput and Validation. In *INFOCOM 2006: 25th IEEE International Conference on Computer Communications*, pages 1–12.

Barré, S., Bonaventure, O., Raiciu, C., and Handley, M. (2010). Experimenting with multipath TCP. *ACM SIGCOMM Computer Communication Review*, 40(4):443–444.

Carns, P., Ligon III, W., Ross, R., and Wyckoff, P. (2005). BMI: A network abstraction layer for parallel I/O. In *19th IEEE International Parallel and Distributed Processing Symposium*, pages 8–pp.

Ford, A., Raiciu, C., Handley, M., Barre, S., and Iyengar, J. (2011). Architectural Guidelines for Multipath TCP Development. *RFC 6182 - Internet Engineering Task Force*.

Ford, A., Raiciu, C., Handley, M., and Bonaventure, O. (2013). TCP Extensions for Multipath Operation with Multiple Addresses. *RFC 6824 - Internet Engineering Task Force*.

Habib, A., Christin, N., and Chuang, J. (2006). Taking Advantage of Multihoming with Session Layer Striping. In *INFOCOM 2006: 25th IEEE International Conference on Computer Communications. Proceedings*, pages 1–6.

Hsieh, H.-Y. and Sivakumar, R. (2002). pTCP: An end-to-end transport layer protocol for striped connections. In *10th IEEE International Conference on Network Protocols*, pages 24–33.

Kerrisk, M. (2010). *The Linux programming interface*, chapter 29 - Threads: Introduction. No Starch Press.

Kimura, B. Y. L., Guardia, H. C., and Moreira, E. S. (2012). Disruption-tolerant sessions for seamless mobility. In *IEEE Wireless Communications and Networking Conference*, pages 2412–2417.

Kimura, B. Y. L., Guardia, H. C., and Moreira, E. S. (2013). A session-based mobile socket layer for disruption tolerance on the internet. *Mobile Computing, IEEE Transactions on*, PP(99):1–14.

Magalhaes, L. and Kravets, R. (2001). Transport level mechanisms for bandwidth aggregation on mobile hosts. In *Network Protocols, 2001. Ninth International Conference on*, pages 165–171.

Raiciu, C., Barre, S., Pluntke, C., Greenhalgh, A., Wischik, D., and Handley, M. (2011a). Improving datacenter performance and robustness with multipath TCP. In *ACM SIGCOMM Computer Communication Review*, volume 41, pages 266–277.

Raiciu, C., Handly, M., and Wischik, D. (2011b). Coupled Congestion Control for Multipath Transport Protocols. *RFC 6356 - Internet Engineering Task Force*.

Raiciu, C., Niculescu, D., Bagnulo, M., and Handley, M. J. (2011c). Opportunistic Mobility with Multipath TCP. In *Proceedings of the Sixth ACM International Workshop on MobiArch*, pages 7–12.

Sivakumar, H., Bailey, S., and Grossman, R. L. (2000). PSockets: The case for application-level network striping for data intensive applications using high speed wide area networks. In *The 2000 ACM/IEEE conference on Supercomputing*, page 37.

Yildirim, E., Yin, D., and Kosar, T. (2009). Balancing TCP Buffer vs Parallel Streams in Application Level Throughput Optimization. In *The Second International Workshop on Data-aware Distributed Computing*, DADC '09, pages 21–30.